C++ STL Competitive Programming Template

Sep 26, 2026 · @Mike

A quick-reference template covering every STL tool you'll actually use in competitive programming. Copy the template, learn the patterns, win the contests.

The Full Template

Copy this at the top of every CP file. It covers fast I/O, common typedefs, and the macros you'll use constantly.

```
#include <bits/stdc++.h>
using namespace std;

// ---------- TYPEDEFS ----------
typedef long long ll;
typedef unsigned long long ull;
typedef pair<int, int> pii;
typedef pair<ll, ll> pll;
typedef vector<int> vi;
typedef vector<ll> vll;
typedef vector<pii> vpii;
typedef vector<string> vs;

// ---------- MACROS ----------
#define pb push_back
#define mp make_pair
#define F first
#define S second
#define all(x) (x).begin(), (x).end()
#define rall(x) (x).rbegin(), (x).rend()
#define sz(x) (int)(x).size()
#define rep(i, a, b) for (int i = (a); i < (b); i++)
#define rrep(i, a, b) for (int i = (a); i >= (b); i--)
#define each(x, a) for (auto& x : a)

// ---------- CONSTANTS ----------
const int MOD = 1e9 + 7;
const int INF = 1e9;
const ll LINF = 1e18;
const double PI = acos(-1.0);
const double EPS = 1e-9;
```

```
// ---------- FAST I/O ----------
void fast_io() {
    ios_base::sync_with_stdio(false);
    cin.tie(NULL);
    cout.tie(NULL);
}

int main() {
    fast_io();
    int t;
    cin >> t;
    while (t--) {
        // solve here
    }
    return 0;
}
```

Containers

Every container you'll reach for in a contest, with the operations that matter.

vector

Dynamic array. Your default container for almost everything.

```
vector<int> v = {3, 1, 4, 1, 5};
v.pb(9);                    // push_back
v.pop_back();               // remove last
v.insert(v.begin() + 2, 7); // insert 7 at index 2
v.erase(v.begin() + 1);     // erase element at index 1
v.resize(10, 0);            // resize to 10, fill new slots with 0
v.clear();                  // empty it
sort(all(v));               // sort ascending
reverse(all(v));            // reverse
int n = sz(v);              // size as int
```

set / multiset

Sorted unique elements (set) or sorted with duplicates (multiset). O(log n) insert/find/erase.

```
set<int> s = {5, 3, 1, 4};
s.insert(2);                // {1, 2, 3, 4, 5}
s.erase(3);                 // {1, 2, 4, 5}
s.count(2);                 // 1 (0 or 1 for set)
auto it = s.find(4);        // iterator to 4
auto it2 = s.lower_bound(3);// first element >= 3
auto it3 = s.upper_bound(3);// first element > 3
int mn = *s.begin();        // smallest
int mx = *s.rbegin();       // largest

multiset<int> ms = {1, 1, 2, 3, 3, 3};
ms.erase(ms.find(3));       // erase ONE 3 (not all of them)
ms.erase(3);                // erase ALL 3s
ms.count(3);                // how many 3s
```

map / unordered_map

Key-value pairs. map is sorted O(log n), unordered_map is hash-based O(1) average.

```
map<string, int> m;
m["alice"] = 5;
m["bob"] = 3;
m.count("alice");            // 1 if exists
m.erase("bob");
for (auto& [key, val] : m)  // structured binding (C++17)
    cout << key << " " << val << "\n";

unordered_map<int, int> um;
um[10] = 100;
// same API, but no ordering — faster for large inputs
// WARNING: worst case O(n) if hash collisions — use map if TLE
```

stack

LIFO. Use for matching brackets, monotonic stack problems, DFS.

```
stack<int> st;
st.push(1);
st.push(2);
st.push(3);
int top = st.top();   // 3
st.pop();             // removes 3
bool empty = st.empty();
```

queue

FIFO. Use for BFS.

```
queue<int> q;
q.push(1);
q.push(2);
int front = q.front(); // 1
int back = q.back();   // 2
q.pop();               // removes 1
```

deque

Double-ended queue. Push/pop from both sides in O(1).

```
deque<int> dq;
dq.push_back(1);
dq.push_front(0);
dq.pop_back();
dq.pop_front();
dq[0];                // random access works
```

priority_queue

Max-heap by default. Use for Dijkstra, greedy problems.

```
priority_queue<int> pq;           // max heap
pq.push(3); pq.push(1); pq.push(5);
int top = pq.top();               // 5
pq.pop();

// min heap
priority_queue<int, vector<int>, greater<int>> minpq;
minpq.push(3); minpq.push(1); minpq.push(5);
int mintop = minpq.top();         // 1

// custom struct
struct Edge {
    int to, cost;
    bool operator>(const Edge& o) const { return cost > o.cost; }
};
priority_queue<Edge, vector<Edge>, greater<Edge>> pq2;
```

pair / tuple

Pack two or more values together. Pairs compare lexicographically (first, then second).

```
pair<int, int> p = {3, 5};
cout << p.F << " " << p.S;     // 3 5
auto p2 = mp(1, 2);             // make_pair

// sorting by pair sorts by first, then second
vector<pii> v = {{3,1}, {1,2}, {3,0}};
sort(all(v)); // {1,2}, {3,0}, {3,1}

// tuple for 3+ values
tuple<int, string, double> t = {1, "hi", 3.14};
auto [a, b, c] = t;            // structured binding
int x = get<0>(t);             // access by index
```

Algorithms

All from `<algorithm>` and `<numeric>` — already included via `bits/stdc++.h`.

Sorting

```
vector<int> v = {5, 3, 1, 4, 2};
sort(all(v));                       // ascending: {1,2,3,4,5}
sort(all(v), greater<int>());        // descending: {5,4,3,2,1}

// custom comparator
sort(all(v), [](int a, int b) {
    return abs(a) < abs(b);          // sort by absolute value
});

// sort structs
struct Player { string name; int pts; };
vector<Player> players;
sort(all(players), [](auto& a, auto& b) {
    return a.pts > b.pts;            // descending by points
});
```

Binary Search

```
vector<int> v = {1, 2, 3, 4, 5, 7, 9}; // MUST be sorted

bool found = binary_search(all(v), 4);      // true

auto it = lower_bound(all(v), 4);           // first >= 4 (points to 4)
auto it2 = upper_bound(all(v), 4);          // first > 4  (points to 5)

int idx = lower_bound(all(v), 4) - v.begin(); // index of first >= 4

// count elements in range [3, 7]
int cnt = upper_bound(all(v), 7) - lower_bound(all(v), 3); // 4
```

Permutations

```
vector<int> v = {1, 2, 3};
do {
    for (int x : v) cout << x << " ";
    cout << "\n";
} while (next_permutation(all(v)));
// prints all 6 permutations: 1 2 3, 1 3 2, 2 1 3, ...
// v must be sorted first to get ALL permutations

prev_permutation(all(v)); // go backwards
```

Min / Max / Accumulate

```
vector<int> v = {3, 1, 4, 1, 5};

int mn = *min_element(all(v));          // 1
int mx = *max_element(all(v));          // 5
auto [lo, hi] = minmax_element(all(v)); // both at once

ll sum = accumulate(all(v), 0LL);       // sum (use 0LL for long long)

int a = 10, b = 20;
int m = min(a, b);                      // 10
int M = max({a, b, 30, 5});             // 30 (initializer list)
```

GCD / LCM (C++17)

```
int g = __gcd(12, 8);        // 4 (works in C++14 too)
int g2 = gcd(12, 8);         // 4 (C++17)
ll l = lcm(12LL, 8LL);      // 24

// GCD of an array
int result = 0;
for (int x : v) result = gcd(result, x);
```

Other Useful Ones

```
vector<int> v = {1, 2, 2, 3, 3, 3, 4};

// unique — removes consecutive duplicates (sort first for all dupes)
sort(all(v));
v.erase(unique(all(v)), v.end());      // {1, 2, 3, 4}

// count and count_if
int c = count(all(v), 3);              // how many 3s
int even = count_if(all(v), [](int x) { return x % 2 == 0; });

// fill
fill(all(v), 0);                       // set everything to 0

// rotate
rotate(v.begin(), v.begin() + 2, v.end()); // move first 2 to the back

// reverse
reverse(all(v));

// nth_element — O(n) partial sort, kth smallest
nth_element(v.begin(), v.begin() + k, v.end()); // v[k] = kth smallest

// iota — fill with 0, 1, 2, 3, ...
vector<int> idx(n);
iota(all(idx), 0);
```

Strings

Strings in C++ are mutable, unlike Java/Python. That makes in-place manipulation fast.

```
string s = "hello world";

// basics
int len = sz(s);                    // 11
char c = s[0];                      // 'h'
s += " bye";                        // append
s.push_back('!');                   // append one char
s.pop_back();                       // remove last char

// substr
string sub = s.substr(6, 5);        // "world" (start, length)

// find
int pos = s.find("world");           // 6 (index), string::npos if not found
if (s.find("xyz") == string::npos)
    cout << "not found";

// replace
s.replace(0, 5, "HI");              // replace first 5 chars with "HI"

// compare
if (s == "hello") { }               // direct comparison
if (s < "world") { }                // lexicographic

// char tricks
char ch = 'A';
bool isLetter = isalpha(ch);
bool isDigit = isdigit(ch);
char lower = tolower(ch);           // 'a'
char upper = toupper('a');          // 'A'
int num = ch - '0';                 // char digit to int ('5' -> 5)
char digit = num + '0';             // int to char digit (5 -> '5')

// string to number / number to string
int n = stoi("42");
ll big = stoll("1234567890123");
double d = stod("3.14");
string ns = to_string(42);

// stringstream (split by spaces, parse mixed input)
stringstream ss("alice 100 bob 200");
string name; int score;
while (ss >> name >> score)
    cout << name << ": " << score << "\n";

// getline (read full line including spaces)
```

```
string line;
getline(cin, line);

// reverse a string
reverse(all(s));

// sort characters
sort(all(s));
```

Bitset & Bit Manipulation

Bitwise ops are O(1) and show up in bitmask DP, subset enumeration, and XOR problems.

Bit Operators

```
int a = 5;    // 101 in binary
int b = 3;    // 011

a & b;        // AND:  001 = 1
a | b;        // OR:   111 = 7
a ^ b;        // XOR:  110 = 6
~a;           // NOT:  ...11111010 (flips all bits)
a << 2;       // left shift:  10100 = 20 (multiply by 4)
a >> 1;       // right shift: 10 = 2 (divide by 2)

// check if ith bit is set
bool isSet = (a >> i) & 1;

// set the ith bit
a |= (1 << i);

// clear the ith bit
a &= ~(1 << i);

// toggle the ith bit
a ^= (1 << i);

// check if power of 2
bool isPow2 = (a > 0) && (a & (a - 1)) == 0;

// count set bits
int bits = __builtin_popcount(a);      // int
int bits64 = __builtin_popcountll(x);  // long long

// lowest set bit
```

```
int lowest = a & (-a);

// log2 (position of highest set bit)
int log2val = __lg(a);  // or 31 - __builtin_clz(a)
```

bitset

Fixed-size bit array. Operations are O(N/64) — 64x faster than a bool array for AND/OR/XOR.

```
bitset<1000> bs;           // 1000 bits, all 0
bs.set(5);                 // set bit 5 to 1
bs.reset(5);               // set bit 5 to 0
bs.flip(5);                // toggle bit 5
bool val = bs[5];          // read bit 5
bool val2 = bs.test(5);    // same but bounds-checked
int cnt = bs.count();      // number of 1s
bool any = bs.any();       // any bit set?
bool none = bs.none();     // all zeros?

bitset<8> a("11001010");
bitset<8> b("10101010");
auto c = a & b;            // AND
auto d = a | b;            // OR
auto e = a ^ b;            // XOR
auto f = a << 2;           // shift left
```

Subset Enumeration (Bitmask)

Iterating over all subsets of n elements — used in bitmask DP.

```
int n = 4;
for (int mask = 0; mask < (1 << n); mask++) {
    // mask represents a subset
    for (int i = 0; i < n; i++) {
        if (mask & (1 << i)) {
            // element i is in this subset
        }
    }
}

// iterate over all submasks of a given mask
for (int sub = mask; sub > 0; sub = (sub - 1) & mask) {
    // sub is a submask of mask
}
```

Common CP Patterns

These patterns combine multiple STL pieces. You'll use them constantly.

Frequency Map

Count how many times each element appears.

```
vector<int> v = {1, 2, 2, 3, 3, 3};
map<int, int> freq;
for (int x : v) freq[x]++;
// freq = {1:1, 2:2, 3:3}

// find most frequent
auto it = max_element(all(freq), [](auto& a, auto& b) {
    return a.second < b.second;
});
cout << it->first << " appears " << it->second << " times";

// for strings
string s = "aabbbcc";
int cnt[26] = {};
for (char c : s) cnt[c - 'a']++;
```

Prefix Sum

Answer range sum queries in O(1) after O(n) preprocessing.

```
vector<int> a = {3, 1, 4, 1, 5, 9};
int n = sz(a);
vector<ll> pre(n + 1, 0);
rep(i, 0, n) pre[i + 1] = pre[i] + a[i];

// sum of a[l..r] (inclusive)
ll range_sum = pre[r + 1] - pre[l];

// 2D prefix sum
vector<vector<int>> grid(n, vector<int>(m));
vector<vector<ll>> pre2d(n + 1, vector<ll>(m + 1, 0));
rep(i, 0, n) rep(j, 0, m)
    pre2d[i+1][j+1] = grid[i][j] + pre2d[i][j+1] + pre2d[i+1][j] - pre2d[i]
[j];
```

Sliding Window

Fixed or variable size window over an array.

```
// fixed window of size k: max sum subarray
int n = sz(a), k = 3;
int cur = accumulate(a.begin(), a.begin() + k, 0);
int best = cur;
rep(i, k, n) {
    cur += a[i] - a[i - k];
    best = max(best, cur);
}

// variable window: longest subarray with sum <= target
int l = 0;
ll sum = 0;
int ans = 0;
rep(r, 0, n) {
    sum += a[r];
    while (sum > target) sum -= a[l++];
    ans = max(ans, r - l + 1);
}
```

Two Pointers

Two indices moving through a sorted array.

```
// two sum on sorted array
sort(all(a));
int l = 0, r = n - 1;
while (l < r) {
    int s = a[l] + a[r];
    if (s == target) { /* found */ break; }
    else if (s < target) l++;
    else r--;
}
```

Custom Comparator for Sorting

```
// sort by second element descending, then first ascending
vector<pii> v = {{1,5}, {2,3}, {3,5}};
sort(all(v), [](pii& a, pii& b) {
    if (a.S != b.S) return a.S > b.S;  // descending by second
    return a.F < b.F;                   // ascending by first
});
// result: {1,5}, {3,5}, {2,3}
```

Fast Input for Large Inputs

When cin is still too slow (rare, but happens on 10^6+ inputs):

```
// already covered by fast_io() in the template
// but if you need even faster:
char buf[1 << 22];
int readInt() {
    int x = 0; char c = getchar_unlocked();
    while (c < '0' || c > '9') c = getchar_unlocked();
    while (c >= '0' && c <= '9') { x = x * 10 + c - '0'; c =
getchar_unlocked(); }
    return x;
}
```